



SpaceSec Shield

Final Technical Report

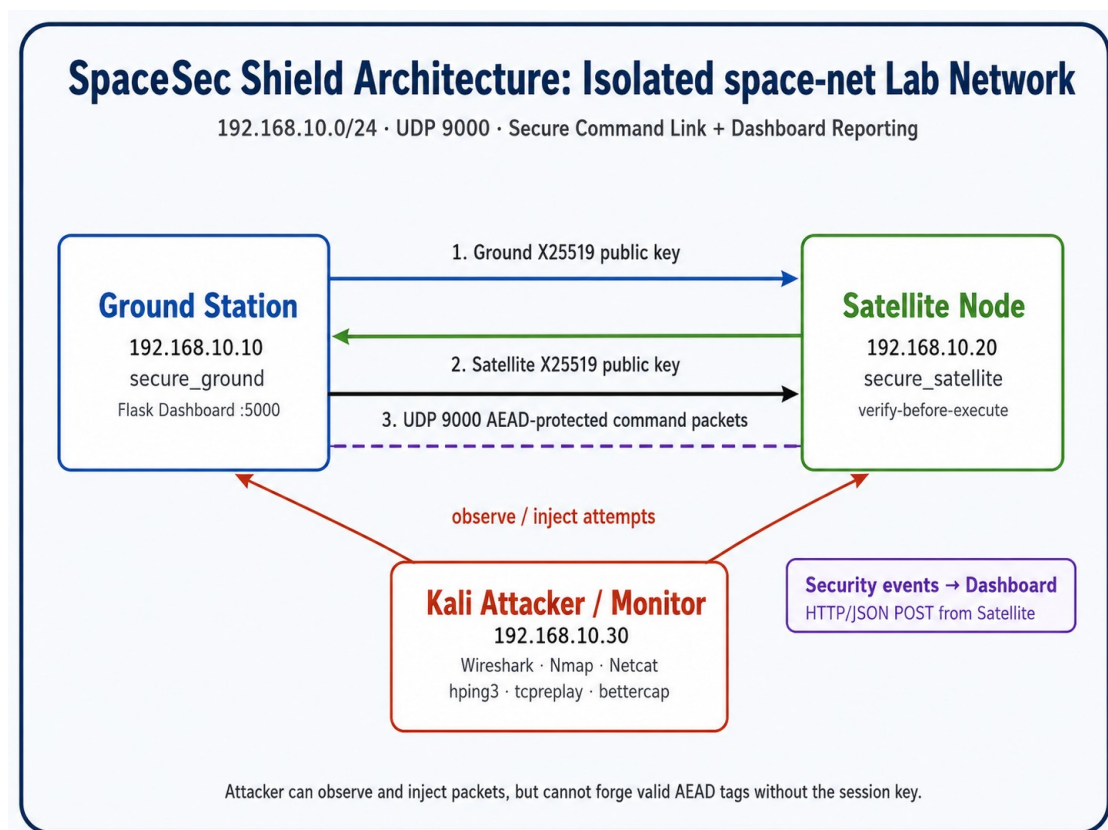
Lightweight Secure Communication and Real-Time Security Monitoring for CubeSat Links

Prepared for: Project Evaluation Committee / NTP 2026

Prepared by: SpaceSec Shield Team

Date: June 2026

Technology Stack: C++ · UDP/BSD Sockets · X25519 ECDH · HKDF-SHA256 · ChaCha20-Poly1305 AEAD · OpenSSL · Linux
tc/netem · Kali Linux · Python Flask · Chart.js



Cover Figure. End-to-end SpaceSec Shield architecture in the isolated three-VM lab



Table of Contents

Abstract

1. Introduction

2. Background and Related Work

3. Design Details

4. Implementation Details

5. Experiments and Results

6. Conclusion and Future Work

References



Abstract

SpaceSec Shield is a secure communications tool. The project deals with the risk of unprotected command links in constrained satellite environments, such as systems with limited power, processing capacity, and bandwidth, where heavyweight enterprise security stacks are unsuitable. The proposed system builds a realistic three-VM and creates a two-way UDP command/response channel. In addition, it adds an authenticated encryption security layer. The validation of the system will be done through penetration testing. Furthermore, the visualization of security decisions through a real-time monitoring dashboard will be done.

The method used is incremental engineering in nature. The establishment of an isolated space-net environment, together with building a stable UDP communication core under simulated delay, jitter and packet loss will be Phase 1 and Phase 2 respectively. The packet protection in Phase 3 is ChaCha20-Poly1305, with HKDF-SHA256 key derivation and ephemeral X25519 ECDH key agreement. Phase 4 assesses command injection, replay, tampering, denial-of-service flooding, man-in-the-middle observation and traffic analysis in the design. Phase 5 implements a Flask and Charts.js dashboard that receives a JSON security event from the Satellite Node which then shows real time commands that are accepted, attacks that are rejected, latency, link status, etc. The results show that authenticated commands were accepted and that forged, replayed, expired, and tampered packets were rejected before execution. The capture of packets confirmed that UDP traffic on port 9000 remained visible, but commands were not exposed in plaintext. The dashboard accurately represented two accepted commands and three critical attack rejections. With heavy flooding, both end-users may not be available when the ECDH handshake is performed due to lack of authenticated peer identity. In general, SpaceSec Shield represents a realistic, transparent, and committee-ready secure communication system for restricted CubeSat-class systems..

Keywords: CubeSat security, lightweight cryptography, ChaCha20-Poly1305, X25519, UDP, anti-replay, penetration testing, security dashboard.

1. Introduction

CubeSats and miniaturized spacecraft cost less and allow faster science but miniaturization restricts processing, memory, power, communications. NASA CubeSat guidance defines a CubeSat as a small satellite standardized to a fixed height, weight and volume, allowing for lower-cost missions to use commercial-off-the-shelf components (NASA CubeSat Launch Initiative, 2017). The same capabilities that bring those advantages bring a cybersecurity challenge. The security controls must protect the mission command without consuming too much resources.

The connection between a ground station and a satellite is essential to a mission. If command data is sent without confidentiality, authentication, integrity protection, and replay resistance, an adversary may observe traffic, inject unauthorized commands, replay previously valid packets, or modify packet fields. Telecommunications dependability, operator confidence, payload conduct, and spacecraft security can all be impacted by such attacks on a space mission. UDP uses low overhead but no connection state, retransmission, delivery guarantees, encryption or authentication are provided by UDP by itself. This makes things worse as the protocol is UDP.

The project, SpaceSec Shield, thus focuses on a realistic but controlled academic objective: to design, implement, validate, and demonstrate a lightweight secure communication layer... The paper is not presented as a full flight-ready space protocol or a complete CCSDS implementation. Security demonstration is not a just security proof; it shows how security properties are proven or verified in the real world during the creation or operation of a given system.

1.1 Problem Statement

The core problem addressed by SpaceSec Shield is the absence of a lightweight, testable, and explainable security layer for a constrained CubeSat communication scenario. The original communication core provided a working two-way UDP command/response channel, but it was intentionally unencrypted. This left the channel vulnerable to eavesdropping, command injection, replay, packet tampering, and misleading operator visibility. The project therefore adds cryptographic protection and monitoring while preserving the low-overhead nature of the UDP link.

1.2 Project Objectives

ID	Objective	Implemented Outcome
O1	Build a reproducible isolated	Three VMs on space-net 192.168.10.0/24.



ID	Objective	Implemented Outcome
	lab for ground-satellite-attacker communication.	
O2	Implement a stable two-way command/response channel.	C++ UDP sender/receiver on port 9000 with timeout handling.
O3	Protect command traffic with lightweight authenticated encryption.	X25519 ECDH, HKDF-SHA256, ChaCha20-Poly1305 AEAD.
O4	Validate security under realistic adversarial scenarios.	Injection, replay, tampering, DoS, MITM observation, traffic analysis.
O5	Make security decisions visible to operators and evaluators.	Flask/Chart.js dashboard with accepted/rejected counters and alert feed.

2. Background and Related Work

2.1 CubeSat Communication Constraints

CubeSat platforms commonly operate with constrained onboard resources. In the context of this project, the relevant constraints are low compute headroom, limited memory, limited power, narrow communication windows, and the need for deterministic behavior. These constraints influenced the decision to avoid heavyweight stacks and to preserve a small UDP-based command channel while adding security at the packet level.

2.2 Space Link Security Standards

The Consultative Committee for Space Data Systems (CCSDS) publishes space communication standards, including the Space Data Link Security Protocol. CCSDS describes this protocol as a structured method for applying data authentication and/or data confidentiality to CCSDS telemetry, telecommand, and advanced orbiting systems data links (CCSDS, 2022). SpaceSec Shield follows the same security goals at a prototype level: it aims to authenticate, encrypt, and validate command packets. However, it does not claim compliance with the full CCSDS standard.

2.3 Cryptographic Foundations

It uses standardized and widely analyzed cryptographic primitives. X25519 is specified in RFC 7748, which provides an elliptic-curve Diffie-Hellman function that takes 32-byte inputs and produces a 32-byte output. The RFC 5869 (Krawczyk & Eronen, 2010) defined HKDF is an HMAC based key derivation function which is suitable for the purpose of deriving two protocol keys from input keying material. According to RFC 8439, ChaCha20 and Poly1305 can be independently used as cipher and MAC respectively. These can also be combined together as a construction of AEAD. After reviewing the available options, we picked OpenSSL because it exposes tested EVP interfaces. The ChaCha20-Poly1305 interface employs a 256-bit key and a 96-bit IV/nonce, supports Additional Authenticated Data, and returns a 128-bit authentication tag (OpenSSL Project). The header metadata that the attacker is able to see is authenticated as AAD according to the packet design of the project. And the command payloads here are encrypted..

2.4 Lightweight Cryptography Context

NIST's plans to develop a standard for lightweight cryptography have highlighted requirements for authenticated encryption, hash and related primitives for constrained devices. The NIST SP 800-232 standard contains Ascon-based lightweight cryptography for constrained devices. The SpaceSec Shield implemented in the prototype uses ChaCha20-Poly1305 instead of Ascon, nevertheless, the design rationale follows lightweight cryptography: offer authenticated confidentiality with limited software cost.

2.5 Related Work Gap

The gap addressed by this project is not the invention of a new cryptographic algorithm. Instead, the contribution is systems integration: a reproducible CubeSat-style testbed, an explainable secure packet format, practical validation against attacks, and a dashboard that makes security outcomes visible. This is suitable for academic evaluation because the committee can inspect the design, run the test scenarios, and observe whether the claimed properties are actually enforced.

3. Design Details

3.1 Overall Architecture

The final architecture consists of three VMs on an isolated network named space-net. The Ground Station sends mission commands, the Satellite Node receives and validates them, and the Kali node monitors or attacks the link. In Phase 5, the Ground Station also hosts the dashboard back end and web interface. The Satellite remains the enforcement point; it decides whether to accept or reject each packet before any command execution occurs.

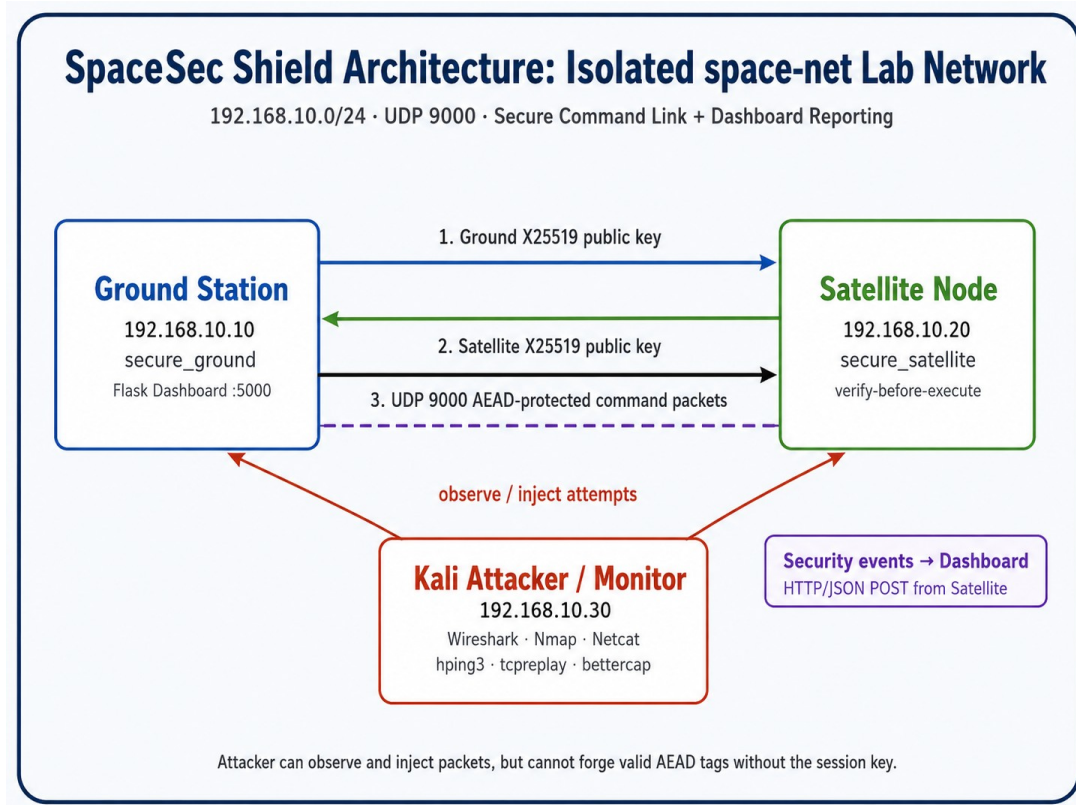


Figure 1. SpaceSec Shield end-to-end architecture: Ground Station, Satellite Node, Kali attacker/monitor, secure UDP channel, and dashboard reporting path.

3.2 Communication Core Design

The communication core makes use of UDP over BSD socket in C++. UDP was chosen due to the fact that it is lightweight, low-overhead and realistic for command-style communication over constrained links. The downside to this is UDP does not guarantee delivery. Sender use a receive timeout in sender to not make the ground station hang when packets are lost. When conditions deteriorate, a lost reply becomes a controlled timeout, not a program crash.

Design Element	Choice
Direction	Two-way command -> response
Protocol	UDP on port 9000
Receiver behavior	Satellite listens continuously and returns ACK for valid commands
Stability control	Ground Station receive timeout prevents indefinite blocking
Space realism	tc/netem delay, jitter, and loss applied on Satellite interface ens33

3.3 Secure Session Establishment

The secure handshake utilizes X25519 key pairs that are ephemeral. The Ground Station sends a 32-bytes X25519 public key to the Satellite Node and Satellite Node responds a 32-bytes public key. The private key from each side is used with the public key of the peer to compute the same ECDH shared secret. HKDF-SHA256



produces a neat 32-byte session key from secret. Session key is never sent out on the network.

3.4 Packet Protection Design

After the handshake, commands are carried in compact protected packets. The header contains a session identifier, timestamp, and nonce. These fields remain visible because the receiver must use them for session and replay checks, but they are authenticated as Additional Authenticated Data. The command payload is encrypted with ChaCha20, and Poly1305 produces the authentication tag. A packet is accepted only if all receiver-side checks pass.

Protected Packet Layout

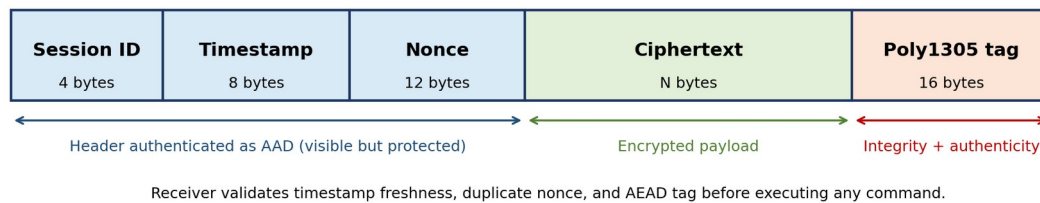


Figure 2. Protected packet layout: authenticated visible header, encrypted payload, and Poly1305 authentication tag.

Field	Size	Security Role
Session ID	4 bytes	Identifies the current secure session; authenticated as AAD.
Timestamp	8 bytes	Supports freshness validation and expiry rejection; authenticated as AAD.
Nonce	12 bytes	Prevents duplicate packet acceptance within a session; authenticated as AAD.
Ciphertext	N bytes	Encrypted command payload.
Poly1305 tag	16 bytes	Verifies integrity and authenticity of AAD and ciphertext.

3.5 Verify-Before-Execute Policy

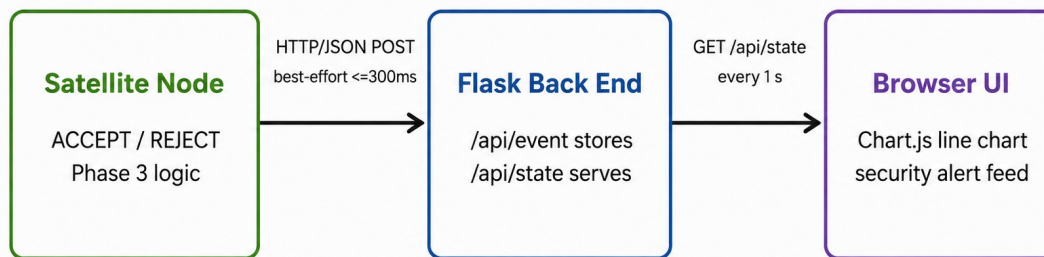
The Satellite Node follows a strict verify-before-execute policy. A command is not executed immediately after reception. Instead, the receiver checks packet format, session state, timestamp freshness, nonce uniqueness, and AEAD tag validity. Only after successful verification is the command passed to the simulated mission logic. This policy is the central security control in the system.

1. Reject malformed packets before cryptographic processing if the format is invalid.
2. Reject expired packets whose timestamps fall outside the freshness window.
3. Reject duplicate nonces already seen in the same session.
4. Reject packets whose Poly1305 tag fails AEAD verification.
5. Execute the command only after all checks pass.

3.6 Monitoring Dashboard Design

The dashboard is intentionally separated from enforcement. The Satellite Node enforces security and then reports each decision to a Flask API. The browser pulls aggregated state once per second and renders accepted/rejected packet counts, average latency, link state, and recent alerts. This avoids duplicating security logic in the dashboard and prevents visualization from becoming a dependency for command validation.

Monitoring Data Flow



The dashboard reports decisions; enforcement remains on the Satellite Node.
Monitoring failure must never stall security enforcement.

Figure 3. Phase 5 monitoring pipeline: Satellite reports decisions to Flask; browser renders live status and alerts.

Reason	Severity	Meaning
OK	LOW	Authenticated command accepted and executed.
BAD_FORMAT	MEDIUM	Malformed or truncated packet, such as raw netcat injection.
REPLAY_TS	CRITICAL	Timestamp outside freshness window.
DUP_NONCE	CRITICAL	Nonce already observed in the same session.
TAG_FAIL	CRITICAL	Poly1305 AEAD tag failure from tampering or forgery.

4. Implementation Details

4.1 Lab Environment

The environment is an isolated VMware lab using a LAN segment named space-net. The static addressing model supports a clear role separation between mission control, satellite, and attacker/monitor nodes. The environment intentionally avoids internet access during validation, except temporary NAT use for package installation.

VM	Role	OS	IP	Main Components
Ground Station	Mission control / secure sender / dashboard host	Ubuntu 22.04 Desktop	192.168.10.10	secure_ground, dashboard.py, templates/index.html, Flask :5000
Satellite Node	Secure receiver / enforcement point	Ubuntu Server 22.04	192.168.10.20	secure_satellite, dashboard_client.cpp, interface ens33
SpaceSec_Kali	Attacker and monitoring node	Kali Linux	192.168.10.30	Wireshark, tcpdump, Nmap, netcat, hping3, tcpreplay, bettercap

4.2 Environment Setup Procedure

1. Import the three OVF virtual machines into VMware Workstation.
2. Recreate the LAN segment named space-net because VMware LAN segments are local to the host machine and do not travel inside the OVF.
3. Attach all three VMs to the same space-net segment.
4. Verify static IPs using ip a.
5. Confirm cross-node connectivity with ping between Ground Station, Satellite Node, and Kali.
6. Run the Satellite receiver first, then start the Ground Station sender.



4.3 Phase 2: Communication Core Implementation

Phase 2 implemented the baseline C++ UDP command/response channel. The Satellite Node binds to UDP port 9000 and listens continuously. The Ground Station sends a command such as CMD: STATUS_CHECK and waits for ACK: CMD: STATUS_CHECK. A two-second receive timeout prevents the sender from hanging when simulated loss removes a packet or reply.

```
# Satellite Node
g++ sat_receiver.cpp -o sat_receiver
./sat_receiver
# Ground Station
g++ ground_sender.cpp -o ground_sender
./ground_sender
```

4.4 Phase 2: Space-Link Simulation

The project uses Linux Traffic Control with netem to emulate space-link impairment. The netem queue discipline can introduce delay, loss, duplication, corruption, and other real-world network impairments, making it suitable for protocol behavior testing. In the lab, commands target the Satellite interface ens33.

```
# Apply delay, jitter, and packet loss on the Satellite Node interface
sudo tc qdisc add dev ens33 root netem delay 500ms 100ms distribution normal loss 10%
# Increase impairment for stress testing
sudo tc qdisc change dev ens33 root netem delay 800ms loss 50%
# Inspect and remove impairment
sudo tc qdisc show dev ens33
sudo tc qdisc del dev ens33 root
```

4.5 Phase 3: Security Layer Implementation

Phase 3 integrated five main C++ modules: handshake handling, cryptographic functions, packet building/parsing, anti-replay logic, and secure sender/receiver logic. OpenSSL libssl and libcrypto were linked during compilation. The final build commands are shown below.

```
g++ -std=c++17 secure_ground_station.cpp handshake.cpp crypto.cpp packet.cpp anti_replay.cpp -o secure_ground -lssl -lcrypto
g++ -std=c++17 secure_satellite_node.cpp handshake.cpp crypto.cpp packet.cpp anti_replay.cpp -o secure_satellite -lssl -lcrypto
```

The Ground Station starts the X25519 exchange, derives a session key using HKDF-SHA256, and sends the protected packets. The Satellite Node receives, verifies, decrypts, and either executes or rejects each packet. Wireshark and tcpdump on Kali validate that the traffic is visible as UDP traffic but does not disclose command plaintext.

4.6 Phase 4: Penetration Testing Implementation

Phase 4 evaluated the security layer from the Kali attacker VM. The toolset included Wireshark and tcpdump for traffic analysis, Nmap for UDP service discovery, netcat for raw packet injection, tcpreplay for replay testing, hping3 for UDP flooding, and bettercap for MITM/ARP-spoofing observation. The tests were performed against UDP port 9000 on the Satellite Node.

```
sudo nmap -sU -p 9000 192.168.10.20
echo "CMD: SELF_DESTRUCT" | nc -u 192.168.10.20 9000
sudo tcpdump -i eth0 udp port 9000 -w capture.pcap
sudo tcpreplay --intf1=eth0 capture.pcap
sudo hping3 --udp -p 9000 -d 120 -E /dev/urandom --flood 192.168.10.20
```

4.7 Phase 5: Dashboard Implementation

Phase 5 implemented the real-time monitoring dashboard. The Flask back end stores events in memory and exposes four routes: POST /api/event, GET /api/state, POST /api/reset, and GET /. The Chart.js front end polls /api/state every second and renders a live chart, stat cards, a status banner, and alert feed.

Route	Method	Purpose
/api/event	POST	Satellite posts one packet-decision event; server stores it and updates counters.
/api/state	GET	Browser requests link status, accepted/rejected counts, latency, and recent events.
/api/reset	POST	Clears in-memory state for a clean demo run.



Route	Method	Purpose
/	GET	Serves the dashboard web page.

The Satellite reports events using a dependency-free C++ HTTP client over a raw TCP socket rather than libcurl. The client uses best-effort semantics and a 300 ms non-blocking timeout. This means that if the dashboard is offline or slow, the Satellite continues processing security decisions and is not stalled by monitoring.

```
g++ -std=c++17 secure_satellite_node.cpp handshake.cpp crypto.cpp packet.cpp anti_replay.cpp dashboard_client.cpp -o  
secure_satellite -lssl -lcrypto
```

5. Experiments and Results

5.1 Validation Strategy

The evaluation combines functional validation, cryptographic validation, adversarial testing, and monitoring validation. Each test case maps directly to a security requirement. The aim is not only to show that the code compiles, but also to demonstrate end-to-end behavior in the isolated three-VM lab.

Area	Question	Expected Evidence
Functional link	Can the Ground Station and Satellite communicate?	Valid command produces ACK; degraded link produces timeout without crash.
Confidentiality	Can an attacker read command plaintext?	UDP packets visible, but command plaintext absent in Wireshark/tcpdump.
Integrity/authenticity	Can an attacker modify or forge commands?	Tampered or forged packets fail AEAD verification and are rejected.
Replay resistance	Can a captured valid packet be reused?	Duplicate nonce and expired timestamp packets are rejected.
Availability	Does the node remain stable under flood?	Satellite remains operational, but network availability can still be degraded.
Operational visibility	Can evaluators see security decisions live?	Dashboard displays accepted and rejected events with severity classification.

5.2 Phase 2 Communication Results

Under normal conditions, the Ground Station sends CMD: STATUS_CHECK and receives ACK: CMD: STATUS_CHECK from the Satellite Node. Under degraded conditions such as delay 800 ms and 50% loss, the system produces a mix of successful ACKs and graceful timeouts. This validates that the communication core remains stable and that packet loss does not cause an indefinite hang.

5.3 Phase 3 Security Results

Test Case	Scenario	Observed Result	Status
TC-01	Secure handshake	Ground and Satellite exchange X25519 public keys and derive a 32-byte session key.	PASS
TC-02	Valid command	Encrypted CMD:CAPTURE_IMAGE is authenticated, decrypted, accepted, and executed safely.	PASS
TC-03	Replay	Resent packet is rejected because the nonce is duplicated.	PASS
TC-04	Tampered payload	Modified ciphertext or tag fails AEAD verification and is rejected.	PASS
TC-05	Expired timestamp	Packet outside freshness window is rejected.	PASS
TC-06	Packet inspection	Kali captures UDP 9000 traffic; plaintext command is not visible.	PASS

The most important Phase 3 conclusion is that the Satellite Node acts as the enforcement point. It accepted the valid authenticated command and rejected replayed, tampered, and expired packets before execution. Packet captures confirmed confidentiality at the traffic-inspection level: the attacker could see UDP traffic, but not the plaintext command.



5.4 Phase 4 Penetration Testing Results

Scenario	Method	Result	Status
Command injection / hijacking	Nmap discovery and netcat raw command injection.	Injected packet had no valid AEAD tag and was rejected; no unauthorized command executed.	PASS
Replay attack	Captured valid UDP packets replayed using tcpreplay.	Replay rejected due to duplicate nonce and timestamp validation.	PASS
Packet tampering	One byte modified in ciphertext and resent.	Poly1305 tag verification failed; packet discarded.	PASS
UDP DoS flooding	hping3 flood to UDP 9000.	Satellite remained operational and rejected invalid/expired packets; availability may still degrade.	PARTIAL PASS
MITM observation	ARP spoofing via bettercap to observe handshake and traffic.	Public ECDH material visible, but session secret not transmitted; passive MITM could not decrypt traffic.	PASS with limitation
Traffic analysis	Wireshark/tcpdump inspection of secure packets.	No plaintext command observed in captured packet payload.	PASS

Phase 4 confirms strong cryptographic protection against injection, replay, and tampering. It also identifies honest boundaries. Cryptography cannot guarantee availability during heavy flooding or jamming; it can reject invalid packets, but it cannot prevent the network from being saturated. In addition, the ECDH handshake provides key agreement but not complete peer identity authentication unless public keys are authenticated through certificates, signatures, or pre-shared keys.

5.5 Phase 5 Monitoring Dashboard Results

Test Case	Action	Observed Result	Status
TC-01	curl POST reachability from Satellite to dashboard.	200 OK and alert appears.	PASS
TC-02	Valid CMD: CAPTURE_IMAGE.	ACCEPTED/OK event; accepted line rises; status CONNECTED.	PASS
TC-03	Replay packet.	REJECTED/DUP_NONCE; CRITICAL alert.	PASS
TC-04	Tampered payload.	REJECTED/TAG_FAIL; CRITICAL alert.	PASS
TC-05	Expired timestamp.	REJECTED/REPLAY_TS; CRITICAL alert.	PASS
TC-06	Second valid CMD: STATUS_CHECK.	Accepted = 2, Rejected = 3; status DEGRADED.	PASS
TC-07	Dashboard offline during Satellite operation.	Satellite keeps running; failed posts return quickly under 300 ms.	PASS
TC-08	Heavy netem delay/loss on ens33.	Monitoring channel degraded because it shares the impaired interface.	NOTED

At the end of the live run, the dashboard state was Accepted = 2, Rejected = 3, average latency approximately 400 ms, and link status = DEGRADED. The three rejected attacks appeared as CRITICAL events: duplicate nonce, tag failure, and replay timestamp. This completes the project narrative: secure enforcement occurs at the Satellite Node, and the dashboard makes the result visible to operators and the committee.

5.6 Results Summary Against Requirements

Requirement	Assessment	Evidence
Confidentiality	Achieved	Command payload is encrypted with ChaCha20-Poly1305; plaintext not visible in packet capture.
Integrity	Achieved	Any ciphertext/header modification invalidates the Poly1305 tag.
Authenticity	Achieved for packet data	Only a holder of the session key can create a valid tag. Peer identity authentication remains future work.
Replay protection	Achieved	Duplicate nonces and expired timestamps are rejected.
Injection resistance	Achieved	Raw forged commands are rejected as invalid format or invalid authentication.
Availability	Partially achieved	System remains stable under flood, but network service can still be degraded.
Monitoring visibility	Achieved	Dashboard visualizes accepted/rejected events and severity classification in real time.



6. Conclusion and Future Work

6.1 Conclusion

SpaceSec Shield demonstrates a lightweight framework for securing communication, allows CubeSat-style links. To begin with an isolated three-virtual machine lab and a well-functioning UDP command/response channel, we add a modern AEAD-based security layer and validate our design via penetration tests. We end with a dashboard exposing our security decisions in real time. It is useful for a committee assessment because it shows the engineering rationale behind each level and the end result.

The strongest technical result is the receiver-side check-then-execute policy. The Satellite Node is suspicious of packet arrival only. It confirms the freshness, uniqueness of nonce, and validity of AEAD tag before executing any command. This policy helps in mitigation command injection, replay, and tampering. The practical cryptographic assurance and low-overhead implementation is achieved with X25519 HKDF-SHA256 ChaCha20-Poly1305.

The system is completed by the dashboard which turns security logs into situational awareness at the operator level. The final demonstration showed that valid commands increased the accepted count, while replay, tampering, and expired timestamp attacks raised CRITICAL alerts immediately. This allows for an explainable and defensible project during discussion.

- The ECDH handshake establishes a shared secret, but peer identity is not fully authenticated. Active key-replacement MITM remains a theoretical limitation without signatures, certificates, or pre-shared public keys.
- Availability under heavy DoS or RF jamming is outside the guarantee of the cryptographic layer. The Satellite can reject invalid packets, but the network can still be saturated.
- The dashboard stores state in memory only. A mission-grade implementation would require persistent logging and forensic retention.
- The Satellite-to-dashboard reporting channel uses plain HTTP inside the isolated lab. A production deployment should authenticate and encrypt monitoring traffic.
- The testbed is virtualized. Real RF hardware, embedded boards, and power/latency measurements would be needed before claiming operational readiness.

6.3 Future Work

- Add handshake identity authenticated using certificates, pre-shared public keys or digital signatures.
- ChaCha20-Poly1305 performance comparison with AES-128-GCM and Ascon-AEAD128, all under similar CPU, memory, latency, power.
- Make the event storage persistent for retrieving dashboard evidence, the audit trails, and post-incident investigation.
- Make sure that the monitoring channel is distinct from the channel that is being compromised, or that it uses its own authenticated secure transport.
- You can limit the rate of messages being sent, filter packets or queue messages to reduce the impact of a DoS attack on UDP 9000. The prototype could be deployed on a lightweight embedded platform such as a Raspberry Pi or possibly a microcontroller-class board to measure practical resource costs.
- The tests should also include fuzzing and malformed handshake messages with spoofed source addresses, and long-session nonce-cache behaviour.
- Implement formal threat modeling utilizing STRIDE or MITRE ATT&CK mappings for space/cyber-physical systems.

References

1. Consultative Committee for Space Data Systems. (2022). Space Data Link Security Protocol (CCSDS 355.0-B-2). <https://ccsds.org/publications/sls/>
2. Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF), RFC 5869. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc5869>
3. Langley, A., Hamburg, M., & Turner, S. (2016). Elliptic Curves for Security, RFC 7748. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7748.html>
4. Linux man-pages project. (2025). tc-netem(8) - Linux manual page. <https://man7.org/linux/man-pages/man8/tc-netem.8.html>



5. NASA CubeSat Launch Initiative. (2017). CubeSat 101: Basic concepts and processes for first-time CubeSat developers. National Aeronautics and Space Administration. https://www.nasa.gov/wp-content/uploads/2017/03/nasa_csl_i_cubesat_101_508.pdf
6. National Institute of Standards and Technology. (2025). SP 800-232: Ascon-Based Lightweight Cryptography Standards for Constrained Devices. <https://csrc.nist.gov/pubs/sp/800/232/final>
7. Nir, Y., & Langley, A. (2018). ChaCha20 and Poly1305 for IETF Protocols, RFC 8439. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc8439>
8. OpenSSL Project. (2025). EVP_chacha20: ChaCha20 and ChaCha20-Poly1305 manual page. https://docs.openssl.org/3.4/man3/EVP_chacha20/
9. SpaceSec Shield Team. (2026a). Environment Import & Setup Guide. Internal project document.
10. SpaceSec Shield Team. (2026b). Phase 2 - Communication Core Development: Lab Build & Handover Documentation. Internal project document.
11. SpaceSec Shield Team. (2026c). Phase 3 Security Layer Final Report. Internal project document.
12. SpaceSec Shield Team. (2026d). Phase 4 Penetration Testing Implementation Report. Internal project document.
13. SpaceSec Shield Team. (2026e). Phase 5 Monitoring Dashboard Implementation & Validation Report. Internal project document.